

Bayesian Machine Learning in Quantitative Finance

Chapter 7: Recovery on Charged-Off Loan Accounts

Based on Mongwe, Mbuva & Marwala

Chapter Outline

- 1 7.1 Introduction
- 2 7.2 Model for Bayesian Detection of Recoveries
- 3 7.3 Markov Chain Monte Carlo Algorithms
 - 7.3.1 Metropolis-Adjusted Langevin Algorithm
 - 7.3.2 Hamiltonian Monte Carlo and the No-U-Turn Sampler
- 4 7.4 Numerical Experiments
- 5 7.5 Results and Discussions
- 6 7.6 Conclusion

The Business Problem: Charge-Offs and Recoveries I

Plain-language setup. A bank lends money. If a borrower stops repaying and the loan is deemed uncollectable through normal servicing, the lender *charges off* (writes off) the loan as a loss on its books. But charging off is an accounting decision, not the end of the story: the lender (or whoever now owns the debt) may still pursue the borrower — letters, calls, payment-plan negotiations, or legal action — and **recover some fraction of the money later**.

The Business Problem in One Sentence

Given everything known about a defaulted account at charge-off, *will there be a meaningful recovery, and how much?* Getting this right lets a bank (or whoever buys the distressed debt) price the loan book correctly and target collections effort efficiently.

The Business Problem: Charge-Offs and Recoveries II

Key Credit-Risk Quantities

- **Recovery rate** = (amount eventually collected – administrative cost of collecting it) / (exposure at default, i.e. the outstanding balance when the loan defaulted).
- **Loss given default (LGD)** = $1 - \text{recovery rate}$: the fraction of the exposure the lender never gets back.
- **Probability of default (PD)** and **exposure at default (EAD)** are the other two pillars of credit-risk modelling; together PD, LGD, EAD drive a bank's regulatory capital requirements (Basel III's internal-ratings-based approach lets a bank estimate LGD itself, which is why getting LGD/recovery modelling right matters commercially, not just academically).

The Business Problem: Charge-Offs and Recoveries III

Why This Is a Hard Modelling Problem

Empirically, the distribution of recovery rates is strongly bimodal/skewed: a large mass of accounts recover (close to) nothing, and a smaller mass recovers (close to) everything, with everything else spread thinly in between. A single smooth regression model tends to fit this poorly – “one size fits all” does not work well here.

The Business Problem: Charge-Offs and Recoveries IV

Prior Work (Non-Bayesian and Bayesian)

- Bellotti & Crook (2012): Tobit and decision-tree models, Beta/fractional-logit transforms of the recovery rate on UK credit-card defaults – plain OLS with macroeconomic variables actually forecast best.
- Calabrese (2014): a mixed continuous-discrete model with a Beta random variable for the continuous part of the recovery rate (Bank of Italy data).
- Loterman et al. (2012): a 24-model benchmark across six bank datasets – non-linear models (e.g. neural networks) beat linear ones.
- Ye & Bellotti (2019): a two-stage Beta-mixture + logistic-regression model that captures the multi-modal recovery-rate shape.
- Zhang & Zhao (2024): the first to use *MCMC sampling* to draw model coefficients and generate LGD distributions – a Bayesian approach, but not one that studies *MCMC algorithm choice* itself.

The Business Problem: Charge-Offs and Recoveries V

This Chapter's Contribution

- First use of **Bayesian logistic regression with an automatic relevance determination prior (BLR-ARD)** to detect the *presence* of recovery on charged-off loans, using real peer-to-peer lending data.
- Being Bayesian means every prediction carries an uncertainty quantification, and the ARD prior automatically ranks which input features matter (with uncertainty on that ranking too) – more interpretable for stakeholders than a black-box point estimate.
- MCMC is used to train the model because it is *asymptotically exact*: unlike variational inference or the Laplace approximation, a properly run MCMC chain converges to the *true* posterior, not an approximation to it.
- Three MCMC samplers are compared for training BLR-ARD, introduced in this chapter for the first time in this modelling context: **MALA**, **HMC**, and the **No-U-Turn Sampler (NUTS)**.

7.2 Bayesian Logistic Regression for Recovery Detection I

Intuition. We want to predict a *binary* outcome – did this charged-off account recover a meaningful amount, yes or no? Logistic regression is the natural workhorse for binary outcomes; the Bayesian wrapper around it gives us a full posterior over the coefficients (hence uncertainty) instead of one point estimate, and the ARD prior additionally tells us which features actually drive that prediction.

Negative Log-Likelihood of the Logistic Model (Eq. 7.1)

$$l(D \mid \mathbf{w}) = \sum_i^N y_i \log(\mathbf{w}^T x_i) + (1 - y_i) \log(1 - \mathbf{w}^T x_i)$$

Symbols: $D = \{\mathbf{x}, y\}$ is the data (features and binary labels); N is the number of observations; $\mathbf{w}$ is the vector of coefficients to be estimated; $y_i \in \{0, 1\}$ is the observed outcome for account i (here: “recovered more than 10%?”); x_i is the feature vector for account i . As usual for logistic regression, $\mathbf{w}^T x_i$ is understood to be passed through the logistic sigmoid $\sigma(z) = 1/(1 + e^{-z})$ to produce a valid probability before taking the logarithm.

7.2 Bayesian Logistic Regression for Recovery Detection II

Un-normalised Log-Posterior (Eq. 7.2)

$$\ln p(\mathbf{w} \mid \mathbf{D}) = l(\mathbf{D} \mid \mathbf{w}) + \ln p(\mathbf{w} \mid \alpha) + \ln q(\alpha)$$

Symbols: $\ln p(\mathbf{w} \mid \alpha)$ is the log-prior on the coefficients given hyperparameters α ; $\ln q(\alpha)$ is the log-distribution of the hyperparameters themselves. This is the quantity MALA/HMC/NUTS will draw samples from (up to a constant), i.e. the target of MCMC.

7.2 Bayesian Logistic Regression for Recovery Detection III

The Automatic Relevance Determination (ARD) Prior

Each coefficient w_i gets a zero-mean Gaussian prior with *its own* variance α_i :

$$w_i \sim \mathcal{N}(0, \alpha_i)$$

Intuition: α_i is a learned “volume knob” per feature. A large α_i lets w_i range far from zero, i.e. the model is allowed to make feature i influential; a small α_i squeezes w_i toward zero, i.e. the model effectively switches feature i off. Because we infer α_i for every feature jointly with $\mathbf{w}$, the model *automatically ranks feature importance* as a by-product of Bayesian inference – no separate feature-selection step is needed.

7.2 Bayesian Logistic Regression for Recovery Detection IV

Jointly Sampling $\mathbf{w}$ and α

The α_i are assumed log-normally distributed with mean zero and unit variance. Rather than alternating between sampling $\mathbf{w}$ and α in a Gibbs scheme, this chapter samples $\mathbf{w}$ and α *jointly* in one MCMC step. This doubles the dimensionality of the space being explored, but empirically gives more stable exploration across all three MCMC methods compared here.

Python: Toy Loan Dataset & BLR-ARD Log-Posterior

```
1 import numpy as np
2
3 # 6 FICTIONAL charged-off accounts: days-since-default, outstanding
4 # balance, and "recovered more than 10%?" (mirrors the book's binary
5 # target definition in Sect. 7.4.2) -- all numbers made up for illustration
6 days_since_default = np.array([30, 400, 90, 250, 600, 150.0])
7 outstanding_balance = np.array([500, 8000, 1200, 4000, 12000, 2500.0])
8 recovered_gt_10pct = np.array([1, 0, 1, 0, 0, 1.0])
9
10 # standardise features, add intercept column
11 x1 = (days_since_default - days_since_default.mean()) / days_since_default.std()
12 x2 = (outstanding_balance - outstanding_balance.mean()) / outstanding_balance.std()
13 X = np.column_stack([np.ones_like(x1), x1, x2])
14 y = recovered_gt_10pct
15
16 def sigmoid(z):
17     return 1.0 / (1.0 + np.exp(-z))
18
19 def log_posterior(w, X, y, prior_var):
20     """Eq. 7.1 (logistic log-lik) + Eq. 7.2 (Gaussian/ARD-style log-prior)."""
21     p = np.clip(sigmoid(X @ w), 1e-9, 1 - 1e-9)
22     loglik = np.sum(y * np.log(p) + (1 - y) * np.log(1 - p))
23     logprior = -0.5 * np.sum(w**2) / prior_var #  $w_i \sim N(0, \alpha_i)$ ,  $\alpha_i = \text{prior\_var}$  here
24     return loglik + logprior
```

Listing 1: A tiny illustrative "loan recovery" dataset and the BLR log-posterior building blocks (author's toy code, not the book's)

Notation You'll Need / Recap

You have already seen MALA and HMC derived in detail (Chapter 3 of this book series). Here we give an *efficient* recap of both, then spend most of our time on what is **new**: the No-U-Turn Sampler (NUTS).

The One-Line Idea Behind Each Method

- **MALA**: random-walk Metropolis-Hastings, but nudged by the gradient of the log-posterior (steepest-ascent drift + noise), corrected by an accept/reject step.
- **HMC**: introduce a fictitious “momentum” variable and simulate deterministic (Hamiltonian) physics for several steps to make a distant, low-autocorrelation proposal, corrected by an accept/reject step.
- **NUTS**: HMC, but with *no user-chosen trajectory length* – the algorithm figures out for itself, on every iteration, how far to simulate before stopping.

7.3.1 MALA – Efficient Recap I

Intuition (quick recap). Plain random-walk Metropolis-Hastings proposes blindly in every direction. MALA instead nudges the proposal *uphill* along the gradient of the log-target before adding noise, so proposed points are more likely to land somewhere probable – fewer wasted, rejected moves.

Langevin Diffusion (Eq. 7.3)

$$d\mathbf{w}_t = \frac{1}{2} f'(\mathbf{w}) dt + dZ_t$$

Symbols: $\mathbf{w}$ are the parameters being sampled; $f'(\mathbf{w})$ is the first derivative (gradient) of the target log-density $f(\mathbf{w})$ w.r.t. $\mathbf{w}$; Z_t is a Brownian-motion process in fictitious time t . This continuous-time process, if we could simulate it exactly, would have the target as its stationary distribution.

7.3.1 MALA – Efficient Recap II

Why the Metropolis Correction Is Needed

The diffusion in Eq. 7.3 usually cannot be solved analytically, so we discretise it (e.g. Euler-Maruyama). Discretisation introduces error that can break detailed balance, so an MH accept/reject step is bolted on afterward to guarantee convergence to the correct target. MALA converges faster than plain MH (thanks to the gradient information) but its samples remain noticeably autocorrelated – this motivates HMC, which takes multiple correlated micro-steps per proposal instead of just one.

Python: From-Scratch MALA Sampler (Toy Running Example)

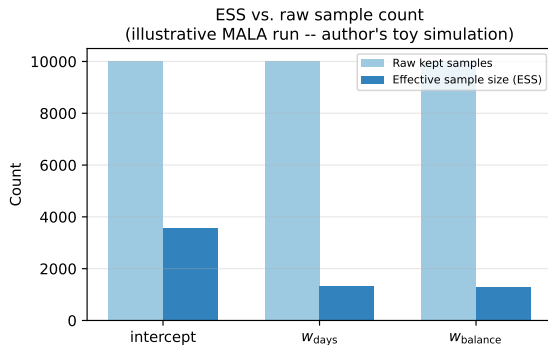
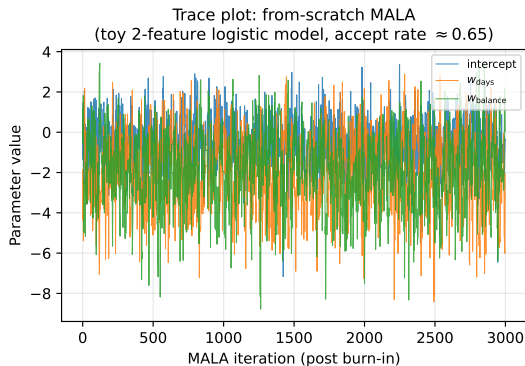
```
1 def grad_log_posterior(w, X, y, prior_var):
2     p = sigmoid(X @ w)
3     grad_loglik = X.T @ (y - p)      # d/dw of Eq. 7.1
4     grad_logprior = -w / prior_var    # d/dw of Gaussian ARD-style prior
5     return grad_loglik + grad_logprior
6
7 def run_mala(X, y, n_iter=12000, burn_in=2000, step=1.6, prior_var=5.0, seed=0):
8     rng = np.random.default_rng(seed)
9     w = np.zeros(X.shape[1])
10    samples, n_accept = np.zeros((n_iter, X.shape[1])), 0
11    for t in range(n_iter):
12        grad = grad_log_posterior(w, X, y, prior_var)
13        # Langevin proposal: drift + Gaussian noise (discretised Eq. 7.3)
14        w_prop = w + 0.5*step**2*grad + step*rng.normal(size=X.shape[1])
15        grad_prop = grad_log_posterior(w_prop, X, y, prior_var)
16        # MH correction accounts for the asymmetric proposal density
17        fwd = w_prop - w - 0.5*step**2*grad
18        bwd = w - w_prop - 0.5*step**2*grad_prop
19        log_alpha = (log_posterior(w_prop, X, y, prior_var) - 0.5*np.sum(fwd**2)/step**2
20                    - log_posterior(w, X, y, prior_var) + 0.5*np.sum(bwd**2)/step**2)
21        if np.log(rng.uniform()) < log_alpha:
22            w = w_prop; n_accept += 1
23        samples[t] = w
24    return samples[burn_in:], n_accept / n_iter    # kept samples, acceptance rate
```

Listing 2: A minimal MALA sampler for the toy BLR posterior of the loan-recovery example

Toy Running Example: MALA Diagnostics

Reminder: this is the author's own illustrative simulation on 6 fictional charged-off accounts – not the book's Lending Club results (those come in Sect. 7.5).

Toy MCMC diagnostics (not the book's Lending Club results)



With a step size tuned toward the book's $\sim 60\%$ MALA target acceptance rate, the trace plots mix but still show visible autocorrelation, and the effective sample size (ESS) is well below the raw number of 17 / 40

7.3.2 HMC – Efficient Recap I

Intuition (quick recap). Treat $-\log(\text{posterior})$ as a landscape of potential energy. Give the parameter a fictitious momentum $\mathbf{p}$ and let it slide around this landscape following the laws of physics (Hamiltonian dynamics) for a while before stopping – because physics conserves energy, the proposal can travel *far* from its start while staying in high-probability territory, dramatically cutting autocorrelation relative to MALA/MH.

Hamiltonian Dynamics (Eq. 7.4)

$$\frac{d\mathbf{w}}{dt} = \frac{\partial H(\mathbf{w}, \mathbf{p})}{\partial \mathbf{p}}, \quad \frac{d\mathbf{p}}{dt} = -\frac{\partial H(\mathbf{w}, \mathbf{p})}{\partial \mathbf{w}}$$

Symbols: $\mathbf{w}$ are the parameters; $\mathbf{p}$ is an auxiliary momentum variable of the same dimension as $\mathbf{w}$; $H(\mathbf{w}, \mathbf{p})$ is the Hamiltonian (total energy) = potential energy (from the target posterior) + kinetic energy (typically a multivariate Gaussian in $\mathbf{p}$).

7.3.2 HMC – Efficient Recap II

Leapfrog Integration (Eq. 7.5) and MH Correction (Eq. 7.6)

$$\mathbf{p}_{t+\epsilon/2} = \mathbf{p}_t + \frac{\epsilon}{2} \frac{\partial H}{\partial \mathbf{w}}, \quad \mathbf{w}_{t+\epsilon} = \mathbf{w}_t + \epsilon \mathbf{M}^{-1} \mathbf{p}_{t+\epsilon/2}, \quad \mathbf{p}_{t+\epsilon} = \mathbf{p}_{t+\epsilon/2} + \frac{\epsilon}{2} \frac{\partial H}{\partial \mathbf{w}}$$

$$\mathbb{P}(\text{accept } \mathbf{w}^*) = \min\left(1, \frac{\exp(-H(\mathbf{w}^*, \mathbf{p}^*))}{\exp(-H(\mathbf{w}, \mathbf{p}))}\right)$$

Symbols: ϵ is the discretisation (leapfrog) step size; $\mathbf{M}$ is a covariance ("mass") matrix in the kinetic-energy term. Two nested steps: (1) leapfrog-integrate the dynamics for some number of steps – this is the *trajectory length*; (2) MH-correct for the leapfrog's numerical error. When trajectory length = 1, HMC collapses exactly to MALA.

7.3.2 HMC – Efficient Recap III

The Tuning Headache HMC Leaves Behind

HMC has two knobs: step size ϵ and trajectory length (number of leapfrog steps L).

- Step size: too small $\rightarrow$ correlated samples, wasted computation; too large $\rightarrow$ large discretisation error, low acceptance. Usually tuned via dual averaging toward a target acceptance rate.
- Trajectory length: too short $\rightarrow$ HMC degenerates into random-walk-like behaviour (losing its whole advantage); too long $\rightarrow$ the simulated path *U-turns* and retraces itself, wasting expensive gradient evaluations without adding new information.

Tuning both well typically needs multiple costly pilot runs. **This is exactly the gap NUTS closes.**

NUTS in Plain Terms

The Core Idea

HMC needs you to choose how many leapfrog steps to simulate – too few wastes the method's power (it barely escapes a random walk), too many wastes computation retracing your steps (the trajectory curves back on itself). **NUTS automatically figures out when to stop** by detecting the moment the simulated path starts “curving back” on itself – a U-turn – so the user never has to hand-pick a trajectory length.

What NUTS Tunes Automatically

- **Step size** ϵ : adapted during an initial burn-in/warm-up phase via primal-dual averaging, targeting a user-specified MH acceptance rate δ .
- **Trajectory length**: adapted *every iteration* by growing the simulated path until it either detects a U-turn or the Hamiltonian numerically diverges.

Result: NUTS performs at least as efficiently as a well-tuned standard HMC, and often better, *without* the costly pilot-tuning runs HMC needs – making it accessible to non-expert users.

Step-Size Adaptation: Primal-Dual Averaging I

Intuition. We want the chain's actual acceptance rate to hover near a target δ (e.g. 0.8) automatically, without manual trial and error, during the burn-in/warm-up window.

Dual-Averaging Update (Eq. 7.7)

$$\epsilon_{t+1} \leftarrow \mu - \frac{\sqrt{t}}{\gamma} \frac{1}{t + t_0} \sum_{i=1}^t H_i, \quad \bar{\epsilon}_{t+1} \leftarrow \eta_t \epsilon_{t+1} + (1 - \eta_t) \bar{\epsilon}_t$$

Symbols: η_t is a decaying adaptation-rate schedule; γ regulates how strongly ϵ_t is pulled toward μ ; H_t is the running difference between the target and actual acceptance rate up to step t ; μ is a free parameter (an anchor value) that ϵ_t gravitates toward; $\bar{\epsilon}_t$ is an averaged, smoothed version of the step size used once warm-up ends.

Step-Size Adaptation: Primal-Dual Averaging II

The Zero-in-Expectation Condition (Eq. 7.8)

$$\mathbb{E}[H_t] = \mathbb{E}[\delta - \alpha_t] = 0$$

Symbols: δ is the target MH acceptance rate; α_t is the actual acceptance probability at step t . This says the step size should be adapted so that, on average, the realised acceptance rate matches the target exactly.

Step-Size Adaptation: Primal-Dual Averaging III

Recommended Settings (used in this chapter, from Hoffman & Gelman, 2014)

$$\mu = \log(10\epsilon_0), \quad \bar{\epsilon}_0 = 1, \quad \bar{H}_0 = 0, \quad \gamma = 0.05, \quad t_0 = 10, \quad \kappa = 0.75, \quad \epsilon_0 = 0.0001$$

These give acceptable performance across a wide variety of target posteriors, and are exactly the settings used later in this chapter's numerical experiments for both HMC and NUTS.

The NUTS Stopping Criterion: The U-Turn / Doubling Procedure

Intuition first. Rather than pre-committing to a trajectory length, NUTS grows a trajectory outward from the current point in a tree-doubling fashion, and stops the instant the path shows signs of turning back toward where it started.

Step-by-Step Outline

- ➊ **Initialise:** start with a single point (current $\mathbf{w}$, freshly sampled momentum $\mathbf{p}$) – a trivial trajectory of length $2^0 = 1$.
- ➋ **Double:** at each round, flip a coin to extend the trajectory either forward or backward in fictitious time, taking that many leapfrog steps to double the trajectory built so far (1, 2, 4, 8, ... steps).
- ➌ **Check for a U-turn:** after each doubling, test whether the vector between the leftmost and rightmost points of the (sub-)trajectory, dotted with the momentum at either end, has become non-positive: $(\mathbf{w}^+ - \mathbf{w}^-) \cdot \mathbf{p}^- \leq 0$ or $(\mathbf{w}^+ - \mathbf{w}^-) \cdot \mathbf{p}^+ \leq 0$. A non-positive value means the path has stopped moving away from its start – it is curving back.
- ➍ **Check for divergence:** also stop doubling if the simulated Hamiltonian has blown up numerically, which signals the step size was too large for this trajectory.
- ➎ **Stop / continue:** stop doubling as soon as step 3 or step 4 triggers anywhere in the (sub-)tree; otherwise keep doubling.
- ➏ **Select the next sample:** from the whole valid trajectory just built, draw the next sample using

Python: Illustrative Leapfrog Step + U-Turn Check

```
1 def leapfrog(w, p, grad_fn, eps):
2     """One leapfrog step implementing Eq. 7.5."""
3     p_half = p + 0.5 * eps * grad_fn(w)
4     w_new = w + eps * p_half # here M = identity for simplicity
5     p_new = p_half + 0.5 * eps * grad_fn(w_new)
6     return w_new, p_new
7
8 def is_u_turn(w_minus, w_plus, p_minus, p_plus):
9     """Implements the U-turn test used inside NUTS' doubling procedure."""
10    delta = w_plus - w_minus
11    return (delta @ p_minus <= 0) or (delta @ p_plus <= 0)
12
13 def grow_trajectory_toy(w0, p0, grad_fn, eps, max_doublings=10):
14     """Illustrative (non-recursive) doubling loop: grow until U-turn."""
15     w_minus, w_plus, p_minus, p_plus = w0, w0, p0, p0
16     trajectory = [w0]
17     for j in range(max_doublings):
18         direction = np.random.choice([-1, 1])
19         n_new_steps = 2 ** j
20         for _ in range(n_new_steps):
21             if direction == 1:
22                 w_plus, p_plus = leapfrog(w_plus, p_plus, grad_fn, eps)
23                 trajectory.append(w_plus)
24             else:
25                 w_minus, p_minus = leapfrog(w_minus, p_minus, grad_fn, -eps)
26                 trajectory.append(w_minus)
27     if is_u_turn(w_minus, w_plus, p_minus, p_plus):
```

7.4.1 Data Description I

Setting: recovery-rate data on charged-off loans from the *Lending Club* peer-to-peer lending platform, 2018 vintage.

Dataset at a Glance

- 658 records total; 70-30 train-test split.
- 43 candidate input variables spanning: balances and utilisation (e.g. total current balance, revolving balance, balance-to-limit ratio), credit-bureau inquiry counts, account counts and delinquency history, loan terms (amount, interest rate, term, instalment, purpose, grade/sub-grade), and borrower demographics (state, employment length, home ownership, annual income).
- The recovery-rate distribution is heavily right-skewed: the large majority of charged-off accounts recover close to nothing, with a secondary small cluster around 10%, and a long thin tail extending out toward full recovery.

7.4.1 Data Description II

Why Not Model the Continuous Recovery Rate Directly (Here)?

Given the highly skewed, near-bimodal shape of the recovery-rate distribution, this chapter frames the task as **binary detection**: will the account recover *more than 10%* of its exposure at default, or not? This turns a hard, multi-modal regression problem into a well-posed Bayesian classification problem – exactly the setting BLR-ARD from Sect. 7.2 was built for.

Python: Toy Version of the 7.4.1 Data-Description Setup

```
1 # The book encodes the target as 1 if recovery rate > 10%, else 0
2 # (Sect. 7.4.2). Here we apply exactly that rule to our toy accounts:
3 recovery_rate = np.array([0.15, 0.02, 0.22, 0.06, 0.01, 0.13]) # toy values
4 target = (recovery_rate > 0.10).astype(float)
5 print(target) # -> [1. 0. 1. 0. 0. 1.] (matches recovered_gt_10pct above)
6
7 # A 70-30 train/test split, mirroring the book's experimental protocol:
8 n = len(target)
9 idx = np.random.default_rng(0).permutation(n)
10 n_train = int(round(0.7 * n))
11 train_idx, test_idx = idx[:n_train], idx[n_train:]
```

Listing 4: Illustrating the book's binary-target construction on the toy loan dataset

7.4.2 Experiment Description

Binary Classification Target

Encode the label as 1 if a defaulted account recovers more than 10% of its exposure at default, else 0. Fit BLR-ARD (Sect. 7.2) to predict this label. 70-30 train-test split.

MCMC Settings (identical protocol for MALA, HMC, NUTS)

- 10,000 total samples generated per method; the first 5,000 are burn-in (used for tuning, then discarded).
- Step sizes tuned via primal-dual averaging (Eq. 7.7–7.8), targeting a 60% acceptance rate for MALA and 80% for both HMC and NUTS.
- HMC's trajectory length (path length) is *fixed* at 25 leapfrog steps – deliberately *not* tuned, to show what NUTS's automatic tuning is being compared against.

7.4.3 Performance Metrics for MCMC Methods

Sampler-Quality Metrics

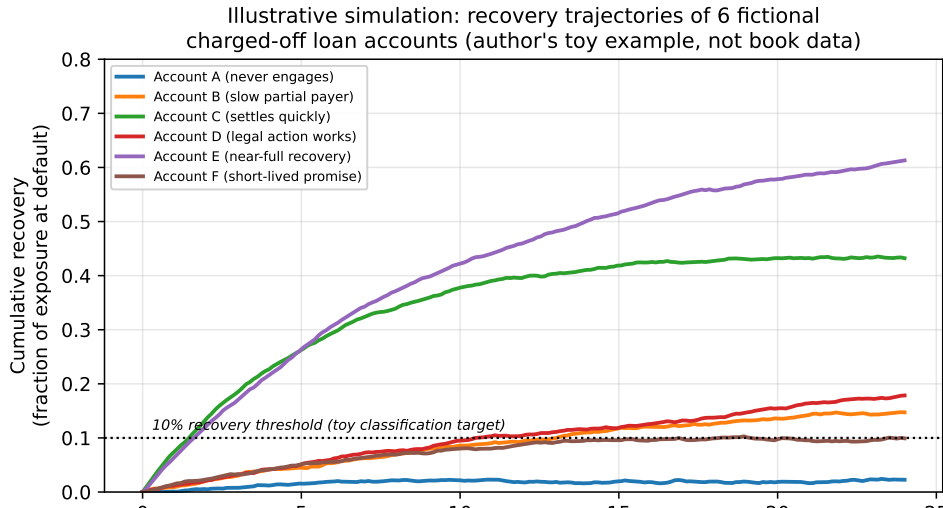
- **Trace plots** of the (un-normalised) target log-posterior – a visual convergence check.
- **Effective sample size (ESS)**: the multivariate ESS estimator of Vats et al. (2019), which accounts for autocorrelation *and* cross-correlation among all sampled parameters jointly (not just one parameter at a time).
- **ESS normalised by execution time (ESS/time)**: because a method that produces more independent-equivalent samples per unit of wall-clock time is more *practically* efficient, even if its raw ESS is lower. This matters because NUTS's extra bookkeeping (tuning the trajectory length on the fly) costs wall-clock time that plain HMC/MALA do not pay.

Predictive-Performance Metric

ROC curve and Area Under the Curve (AUC): true-positive rate vs. false-positive rate on held-out test data. $AUC \approx 0.5$ means no better than chance; $AUC \rightarrow 1$ means near-perfect ranking. AUC is a natural choice here because the relative cost of a false positive vs. a false negative in recovery detection is not clearly known up front.

Toy Running Example: Recovery Trajectories

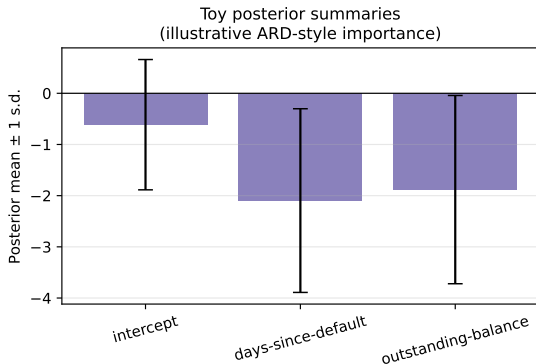
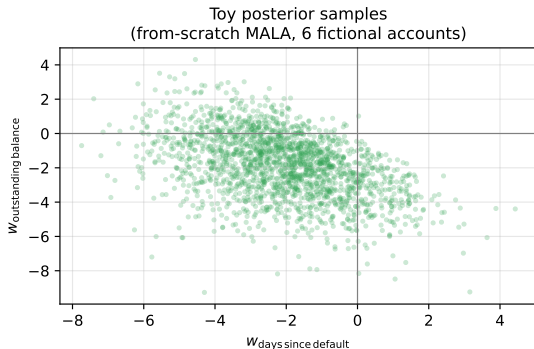
Illustrative simulation only – to build intuition for what "recovery over time" looks like before we look at the book's real classification results.



Toy Running Example: Posterior Summaries

Fitting the toy BLR model (Sect. 7.2 code) with the from-scratch MALA sampler (Sect. 7.3.1 code) to our 6 fictional accounts gives the following posterior:

Toy running example: posterior of a Bayesian logistic model for
'recovered > 10%?' fit to 6 fictional charged-off accounts



With only 6 (fictional) data points the posterior is understandably wide – exactly the kind of honest

Sampler Efficiency: Effective Sample Size

Raw ESS vs. Time-Normalised ESS (book's Lending Club experiments)

- **Raw ESS:** NUTS > HMC > MALA (roughly 2,500 vs. 1,250 vs. under 250 out of 2,500 post-burn-in samples). NUTS produces the least autocorrelated samples of the three, exactly as its adaptive trajectory length is designed to achieve.
- **Time-normalised ESS (ESS/time):** the ranking *flips* – MALA > HMC > NUTS. MALA has by far the lowest execution time of the three (no Hamiltonian dynamics to simulate), so per unit of wall-clock time it is the most efficient, despite having the worst raw ESS.
- Execution-time ranking: MALA fastest, then HMC, then NUTS slowest – NUTS pays extra wall-clock cost for the on-the-fly trajectory-length tuning (the doubling procedure) on every single iteration.

Key Takeaway

There is no free lunch: NUTS wins on sample quality per draw, MALA wins on sample quality per second. Which matters more depends on whether wall-clock time or total iteration budget is the binding constraint in practice.

Convergence and Predictive Performance

Convergence

The negative log-likelihood trace plots for all three MCMC methods overlap in the same stable range across thousands of samples – all three methods have converged to (and are exploring) the same target posterior.

Predictive Performance (Test-Set AUC)

- MALA: $\text{AUC} = 0.6535$ (highest).
- NUTS: $\text{AUC} = 0.6442$.
- HMC: $\text{AUC} = 0.6424$ (HMC and NUTS are essentially tied).

Interestingly, MALA achieves the *best* predictive performance despite having the *worst* raw ESS – a reminder that "more effective samples" does not automatically translate into "better point predictions" on a held-out set; posterior predictive averaging can be fairly forgiving of a moderately correlated chain.

Feature Importance: What Drives Recovery? I

Table 7.1 – Top-5 Ranked Features per Method (by posterior α_i)

Rank	MALA	HMC	NUTS
1	acc-open-past-24mths	grade	total-bal-il
2	num-actv-rev-tl	purpose	application-type
3	addr-state	home-ownership	annual-inc
4	tot-cur-bal	open-acc	num-rev-accts
5	sub-grade	title	all-util

The Common Thread Across All Three Methods

Despite ranking different features #1, all three MCMC methods' top-10 lists repeatedly surface the same handful of themes: *loan purpose*, *title*, *home-ownership status*, *interest rate*, *number of (revolving/bank) accounts*, and *grade-related variables*. The three samplers broadly agree on which features matter, even though they disagree on the exact ranking order.

Feature Importance: What Drives Recovery? II

Does This Make Sense?

- **Purpose:** a loan taken out for debt consolidation plausibly signals a borrower already under financial strain – higher default risk, and (conditional on default) lower odds of meaningful recovery – versus, say, a home-improvement loan.
- **Interest rate:** higher rates are charged to riskier borrowers, who are correspondingly less likely to have the means to repay after default.
- **Home ownership / grade:** proxies for overall financial stability and creditworthiness that plausibly correlate with post-default collectability.

This qualitative face-validity is itself a useful sanity check on an otherwise “black-box-feeling” ARD ranking.

7.6 Conclusion I

What This Chapter Did

- First-in-literature application of a Bayesian inference framework (BLR-ARD) to detect *presence* of recovery on charged-off peer-to-peer lending accounts.
- Trained the same model with three MCMC samplers – MALA, HMC, and NUTS – under a matched experimental protocol.

7.6 Conclusion II

What Was Found

- **Speed:** MALA fastest, HMC in the middle, NUTS slowest (extra cost of on-the-fly trajectory tuning).
- **Raw ESS:** NUTS highest, HMC second, MALA lowest.
- **Time-normalised ESS:** MALA wins, NUTS is the laggard – the exact opposite ranking to raw ESS.
- **Predictive performance (AUC):** MALA best, HMC and NUTS statistically similar to one another.
- **Feature relevance:** all three MCMC methods broadly agree that *purpose*, *home-ownership*, and *interest rate* are among the most important drivers of whether a charged-off account will show recovery.

7.6 Conclusion III

Limitations and Future Work (as noted by the authors)

- This chapter only detects *whether* recovery happens (binary), not *how much* is recovered – modelling the actual continuous recovery amount with Bayesian techniques remains unexplored in the literature and is flagged as future work.
- More complex models (e.g. deep neural networks) could be substituted for BLR-ARD within the same Bayesian/MCMC training framework.

Looking Ahead

The next chapter turns to Bayesian evidence metrics based on normalizing flows, applied to selecting and comparing models that predict audit outcomes for South African municipalities.